

Guía del Ejemplo: Clasificación SVM con Kernel RBF en MATLAB

Autor: Prof. D.Sc. BARSEKH-ONJI Aboud **Institución:** Facultad de Ingeniería, Universidad Anáhuac México **Curso:** Redes Neuronales y SVM **Contacto:** aboud.barsekh@anahuac.mx **ORCID:** 0009-0004-5440-8092

Introducción

Este documento explica paso a paso el script MATLAB `SVM_ejemplo_clasificacion.m`, que implementa un flujo completo de clasificación binaria con SVM de kernel RBF. Se utiliza el dataset clásico `fisheriris` de Fisher (1936), restringido a dos clases (*Iris versicolor* y *Iris virginica*) y dos características (longitud y ancho del pétalo), lo que permite visualizar directamente la frontera de decisión en dos dimensiones.

El objetivo didáctico es que el estudiante pueda ver con claridad:

1. Cómo se divide el espacio en regiones de decisión
 2. Qué puntos se convierten en vectores de soporte y por qué
 3. El efecto del parámetro C sobre el margen y los errores
 4. Cómo se evalúa cuantitativamente el desempeño de un clasificador
-

1. Dataset: Iris de Fisher

El dataset `fisheriris` contiene 150 muestras de tres especies de iris con cuatro características medidas en centímetros: longitud del sépalo, ancho del sépalo, longitud del pétalo y ancho del pétalo.

En este ejemplo se usan:

- **Clases:** `versicolor` (muestras 51–100, etiqueta -1) y `virginica` (muestras 101–150, etiqueta $+1$)
- **Características:** columnas 3 y 4 (longitud y ancho del pétalo), que son las más discriminativas para separar estas dos especies

Característica	Variable MATLAB	Descripción
x_1	<code>X(:,1)</code>	Longitud del pétalo (cm)
x_2	<code>X(:,2)</code>	Ancho del pétalo (cm)
y	<code>Y</code>	Clase: <code>versicolor</code> o <code>virginica</code>

El subconjunto `versicolor/virginica` **no es perfectamente separable linealmente**, lo que lo hace ideal para demostrar el Soft Margin SVM con kernel RBF.

2. Preparación de los Datos

2.1 Carga y selección

```
load fisheriris;
X = meas(51:end, 3:4);
Y = categorical(species(51:end));
```

`meas` es una matriz 150×4 . Se extraen las filas 51 a 150 (las dos clases de interés) y las columnas 3 y 4. `species` es un arreglo de cadenas; `categorical` lo convierte en variable categórica, que es lo que requiere `fitcsvm`.

2.2 Partición entrenamiento/prueba

```
cv = cvpartition(Y, 'HoldOut', 0.3);
```

`cvpartition` con opción 'HoldOut' divide los datos en dos conjuntos: 70% para entrenamiento y 30% para prueba, con **muestreo estratificado** (se preserva la proporción de cada clase). Esto es importante: una partición aleatoria no estratificada podría dejar una clase sub-representada en el entrenamiento.

¿Por qué 70/30? Es una convención empírica ampliamente aceptada para datasets de tamaño moderado ($N < 1000$). Para datasets pequeños se recomienda validación cruzada completa.

2.3 Normalización

```
X_train_std = (X_train - mu) ./ sigma;
X_test_std = (X_test - mu) ./ sigma;
```

La normalización **z-score** transforma cada característica para que tenga media 0 y desviación estándar 1. Es **obligatoria** en SVM porque:

- El kernel RBF $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$ depende de la distancia euclidiana entre puntos
- Si una característica tiene valores en el rango $[0, 1000]$ y otra en $[0, 1]$, la primera dominará artificialmente la distancia
- La normalización pone todas las características en la misma escala de importancia

Advertencia crítica: Los parámetros de normalización (μ y σ) se calculan **solo con los datos de entrenamiento** y luego se aplican al conjunto de prueba. Usar los datos de prueba en la normalización constituye *data leakage* (filtración de información) y produce estimaciones de desempeño demasiado optimistas.

3. Entrenamiento del Modelo SVM

3.1 La función fitcsvm

```
svmModel = fitcsvm(X_train_std, Y_train, ...  
    'KernelFunction', 'rbf', ...  
    'BoxConstraint', 1, ...  
    'KernelScale', 'auto', ...  
    'Standardize', false);
```

Los parámetros clave:

Parámetro	Valor	Significado
'KernelFunction'	'rbf'	Kernel gaussiano $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \ \mathbf{x}_i - \mathbf{x}_j\ ^2)$
'BoxConstraint'	$C = 1$	Parámetro de regularización del Soft Margin
'KernelScale'	'auto'	MATLAB estima σ (y por tanto $\gamma = 1/(2\sigma^2)$) automáticamente con una heurística basada en la mediana de distancias entre pares
'Standardize'	false	Ya normalizamos manualmente; no volver a normalizar

La heurística de MATLAB para 'auto' es $\sigma = \text{mediana}(\|\mathbf{x}_i - \mathbf{x}_j\|)$ sobre una muestra de pares, que es un punto de partida razonable.

3.2 Información del modelo entrenado

Tras el entrenamiento, el objeto `svmModel` contiene:

- `svmModel.SupportVectors`: coordenadas de los vectores de soporte (en el espacio normalizado)
- `svmModel.IsSupportVector`: vector lógico de longitud N_{train} , verdadero si el punto es VS
- `svmModel.Alpha`: valores $|\alpha_i^* - \alpha_i^{**}|$ de los vectores de soporte
- `svmModel.Bias`: valor de b^*

4. Evaluación del Modelo

4.1 Predicción y matriz de confusión

```
Y_pred = predict(svmModel, X_test_std);  
confMat = confusionmat(Y_test, Y_pred);
```

La **matriz de confusión** es una tabla $K \times K$ donde la entrada (i, j) es el número de muestras de la clase verdadera i que fueron predichas como clase j :

$$\mathbf{C} = \begin{pmatrix} VP & FN \\ FP & VN \end{pmatrix}$$

donde VP = verdaderos positivos, VN = verdaderos negativos, FP = falsos positivos, FN = falsos negativos.

4.2 Métricas de desempeño

Del script, las métricas calculadas son:

Métrica	Fórmula	Interpretación
Exactitud (<i>Accuracy</i>)	$(VP + VN)/N_{test}$	Fracción de predicciones correctas
Precisión (<i>Precision</i>)	$VP/(VP + FP)$	De los que predije como positivos, ¿cuántos lo son?
Sensibilidad (<i>Recall</i>)	$VP/(VP + FN)$	De los positivos reales, ¿cuántos detecté?
F1-Score	$2 \cdot (P \cdot R)/(P + R)$	Media armónica de precisión y recall

En problemas balanceados, la exactitud es suficiente. En problemas desbalanceados, se prefiere F1-Score o la curva ROC.

5. Búsqueda de Hiperparámetros

5.1 ¿Por qué es necesario ajustar C y γ ?

Con los valores por defecto ($C = 1$, $\gamma = \text{auto}$), el modelo puede no ser óptimo. La selección de hiperparámetros es el paso más crítico en la práctica de SVM.

5.2 Búsqueda en grilla logarítmica

```
C_grid = logspace(-2, 3, 12);  
gamma_grid = logspace(-4, 1, 12); % gamma: anchura del kernel RBF
```

Se usan escalas logarítmicas porque el comportamiento de SVM es relativamente estable dentro de órdenes de magnitud, pero cambia drásticamente entre órdenes distintos.

Para cada par (C, γ) , se entrena un SVM con validación cruzada de 5 particiones:

```
mdl = fitcsvm(X_train_std, Y_train, ...
    'KernelFunction', 'rbf', ...
    'BoxConstraint', C, ...
    'KernelScale', 1/sqrt(2*g), ...
    'Standardize', false, ...
    'CrossVal', 'on', ...
    'KFold', 5);
acc = 1 - kfoldLoss(mdl);
```

La relación entre `KernelScale` (σ) y γ es: $\gamma = \frac{1}{2\sigma^2}$, por tanto $\sigma = \frac{1}{\sqrt{2\gamma}}$.

El resultado es un mapa de calor donde cada celda muestra la exactitud de validación cruzada para un par (C, γ) . El par óptimo es el de mayor exactitud.

6. Visualización de la Frontera de Decisión

6.1 Construcción de la malla

```
h = 0.05;
[x1g, x2g] = meshgrid(x1_min:h:x1_max, x2_min:h:x2_max);
Xgrid = [x1g(:), x2g(:)];
[~, scores] = predict(svmModel_opt, Xgrid);
```

Se crea una malla densa de puntos sobre el espacio de características. Para cada punto de la malla, se calcula el **score** (valor continuo de la función de decisión antes del **sign**). El signo del score determina la clase predicha; su magnitud determina la confianza.

6.2 Interpretación visual

La visualización genera tres regiones:

- **Región azul claro:** predicción = `versicolor`
- **Región rojo claro:** predicción = `virginica`
- **Frontera entre regiones:** la curva de nivel donde el score = 0, es decir, el hiperplano de decisión en el espacio original

Los **vectores de soporte** se marcan con un símbolo distinto (diamante) porque son los únicos puntos que determinan la frontera. Puntos de entrenamiento que no son vectores de soporte podrían eliminarse sin cambiar la frontera.

7. SVR: Extensión a Regresión

El script también incluye un ejemplo de ϵ -SVR usando el dataset `carsmall` (datos de automóviles estadounidenses).

- **Entradas (X):** potencia del motor (*Horsepower*) y peso del vehículo (*Weight*)
- **Salida (y):** consumo en millas por galón (*MPG*)

```
svrModel = fitrsvm(X_reg_std, y_reg, ...  
    'KernelFunction', 'rbf', ...  
    'BoxConstraint', 10, ...  
    'Epsilon', 0.5, ...  
    'KernelScale', 'auto', ...  
    'Standardize', false);
```

El parámetro **Epsilon** define el ancho del tubo de tolerancia: residuos menores que $\epsilon = 0.5$ mpg no se penalizan. Esta elección es específica del dominio: un error de medio mpg es prácticamente irrelevante para el objetivo de este modelo.

El desempeño se evalúa con RMSE (*Root Mean Square Error*):

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

Y con R^2 (coeficiente de determinación):

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

donde $\bar{y}$ es la media de los valores reales. Un $R^2 \approx 1$ indica que el modelo explica casi toda la varianza de los datos.

8. Resumen del Flujo de Trabajo

```
Datos -> Particion (70/30, estratificada)  
      -> Normalizacion z-score (solo con entrenamiento)  
      -> Entrenamiento SVM (fitcsvm, kernel RBF)  
      -> Evaluacion preliminar (accuracy, confusion)  
      -> Busqueda de hiperparametros (grilla C x gamma, CV 5-fold)  
      -> Re-entrenamiento con parametros optimos  
      -> Evaluacion final en conjunto de prueba  
      -> Visualizacion (frontera de decision, vectores de soporte)
```

9. Conceptos Clave del Script

Concepto	Función MATLAB	Descripción
Partición datos	<code>cvpartition</code>	División estratificada entrenamiento/prueba
Normalización	Operaciones matriciales	z-score manual para control explícito
Entrenamiento SVM	<code>fitcsvm</code>	Soft Margin SVM con kernel elegible
Predicción	<code>predict</code>	Aplica el modelo entrenado a nuevos datos
Evaluación	<code>confusionmat</code>	Matriz de confusión
Vectores de soporte	<code>IsSupportVector</code>	Índice lógico en el modelo
Búsqueda de hiperparámetros	<code>kfoldLoss</code>	Error de validación cruzada
SVR	<code>fitrsvm</code>	Regresión con SVM
Métricas de regresión	Cálculo manual	RMSE, R^2

10. Ejercicios Propuestos

1. **Efecto de C :** Entrena tres modelos SVM con $C \in \{0.01, 1, 1000\}$ usando el mismo γ . Compara el número de vectores de soporte y la exactitud en prueba. ¿Qué observas?
2. **Comparación de kernels:** Repite el experimento usando kernel lineal y polinomial de grado 3. Visualiza las tres fronteras de decisión en la misma figura. ¿Cuál es más suave?
3. **Todas las características:** Usa las cuatro columnas de `meas` en lugar de solo las dos del pétalo. ¿Mejora la exactitud? ¿Por qué no puedes visualizar la frontera de decisión en este caso?
4. **Problema multi-clase:** Extiende el ejemplo a las tres especies de iris. Usa `fitcecoc` con la opción '`Learners`', `templateSVM(...)` para construir un clasificador OvO. Reporta la matriz de confusión de 3×3 .
5. **SVR con más variables:** Agrega la columna de cilindros (`Cylinders`) como tercera entrada al modelo SVR. ¿Mejora el RMSE? Justifica.

Documento preparado como material de apoyo para el curso de Redes Neuronales y SVM, Facultad de Ingeniería, Universidad Anáhuac México.